



移位异或纠删码的
高效实现研究

孙铎

目录

研究内容与进度

已完成工作与结果

后期工作、困难与可行性

移位异或纠删码的高效实现研究

中期报告

学生：孙铎

导师：王强 副教授

哈尔滨工业大学（深圳）计算机科学与技术学院

2026 年 6 月 18 日



汇报提纲

移位异或纠删码的
高效实现研究

孙铎

目录

研究内容与进度

已完成工作与结果

后期工作、困难与可行性

① 研究内容与进度

② 已完成工作与结果

③ 后期工作、困难与可行性



研究内容与进度

先看一个真实问题：存储系统会丢块

移位异或纠删码的
高效实现研究

孙铎

目录

研究内容与进度

已完成工作与结果

后期工作、困难与可行性

为什么不能只靠三副本

- 云存储、文件系统和对象存储都要容忍磁盘或节点失效
- 三副本实现简单，但 1TB 数据要占约 3TB 空间
- 纠删码把数据切成 k 个数据块，再生成 m 个校验块，用更低冗余恢复丢失块

本课题关心的具体场景

- 存储系统中同时可能丢数据块，也可能丢校验块
- 编码和恢复速度越快，系统写入、修复和降级读越不容易被拖慢
- 目标是在 Ceph 这类真实系统里做出高吞吐纠删码实现

一句话概括

少占空间，同时在块丢失时尽快把数据和校验都恢复回来。



研究内容与进度

为什么选择移位异或码

移位异或纠错码的
高效实现研究

孙铎

目录

研究内容与进度

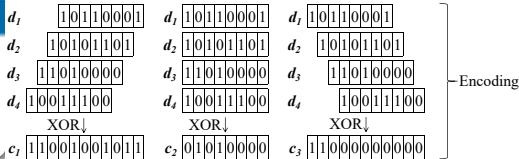
已完成工作与结果

后期工作、困难与可行性

传统 Reed-Solomon 码

- 可靠、成熟，Ceph 等系统已广泛使用
- 编解码依赖有限域矩阵运算
- 高性能实现优化复杂

例：4 个数据块经过不同偏移后异或生成 3 个校验块



Two-tone 移位异或码

- 校验块由数据块移位后异或得到
- 移位可用地址偏移实现
- 理论上很快，但缺少工程级高性能实现

孙铎

移位异或纠错码的高效实现研究

哈尔滨工业大学（深圳）计算机科学与技术学院

4 / 19



研究内容与进度

中期已经做出了什么

移位异或纠删码的
高效实现研究

孙铎

目录

研究内容与进度

已完成工作与结果

后期工作、困难与可行性

已经完成的系统

- 完成 Two-tone 编码、解码和修复流程分析
- 实现约 2000 行 C++ FastTT 核心库
- 把 FastTT 做成 Ceph v18.2.7 纠删码插件
- 通过 Ceph 接口测试和集群端到端读写验证

已经得到的结果

- 相较 ISA-L: 编码平均提升 59.1%
- 相较 ISA-L: 解码平均提升 50.9%
- 单块丢失恢复最高提升 182.1%
- 与学术基线 Cerasure 对比也取得整体优势
- 相关成果已整理为论文并投稿 IPDPS 2026

孙铎

移位异或纠删码的高效实现研究

哈尔滨工业大学（深圳）计算机科学与技术学院

5 / 19



已完成工作与结果

Two-tone 如何编码和恢复

移位异或纠删码的
高效实现研究

孙铎

目录

研究内容与进度

已完成工作与结果

后期工作、困难与可行性

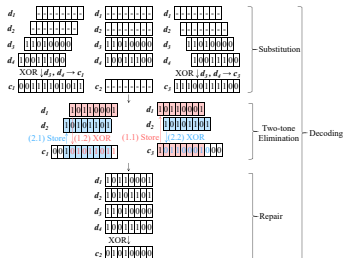
编码时做什么

- 数据被切成 k 个数据块
- 每个校验块由数据块移位后异或得到
- 移位靠地址偏移完成，核心计算是异或

恢复时做什么

- 先把还活着的数据代入校验块
- 按 Two-tone 顺序恢复丢失数据
- 如果校验块也丢了，同步补回来

解码示例：丢失两个数据块和一个校验块





已完成工作与结果

慢在哪里：不是异或慢，而是反复搬数据

移位异或纠错码的
高效实现研究

孙铎

目录

研究内容与进度

已完成工作与结果

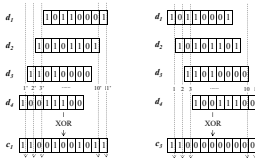
后期工作、困难与可行性

直观实现会做很多重复访问

- 编码时，一个数据块会分散写入多个校验块
- 解码时，数据恢复和校验修复被分成两段流程
- 同一批校验块会被多轮读写，缓存局部性差



合并前的两种访问方式都有明显瓶颈



横向遍历：数据连续，校验分散写

纵向遍历：校验集中，数据反复读

FastTT 的判断

Two-tone 的异或计算很轻，真正拖慢吞吐的是内存访问次数、访问顺序和缓存命中率。



已完成工作与结果

FastTT 做了四个具体改动

移位异或纠删码的
高效实现研究

孙铎

目录

研究内容与进度

已完成工作与结果

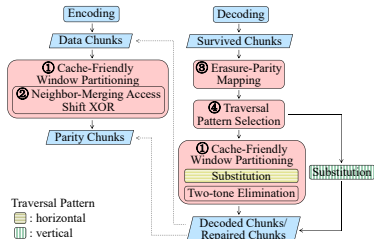
后期工作、困难与可行性

对应四个瓶颈

- 校验修复多跑一遍 $\Rightarrow$ 边解码边修校验
- 长块跨来跨去访问 $\Rightarrow$ 切成缓存窗口
- 校验块写入太分散 $\Rightarrow$ 相邻数据先合并
- 单块丢失最常见 $\Rightarrow$ 单遍直接恢复

落地形态

全部集成进 FastTT, 并以 Ceph 插件完成系统验证。



编码侧优化写入路径；解码侧同时减少冗余流程和常见故障开销。



已完成工作与结果

边恢复数据，边修复丢失校验

移位异或纠删码的
高效实现研究

孙铎

目录

研究内容与进度

已完成工作与结果

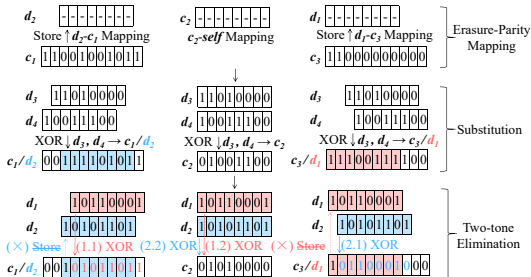
后期工作、困难与可行性

原来的问题

解码出数据后再单独重编码修复校验块，会多访问一轮数据。

FastTT 的做法

- 把丢失数据块映射到存活校验块
- 把丢失校验块映射到自身结构
- 每恢复一个码字就同步更新
- 解码吞吐提升约 100.2%



孙铎

移位异或纠删码的高效实现研究

哈尔滨工业大学（深圳）计算机科学与技术学院

9 / 19



已完成工作与结果

把长块切成窗口，让缓存真正用起来

移位异或纠删码的
高效实现研究

孙铎

目录

研究内容与进度

已完成工作与结果

后期工作、困难与可行性

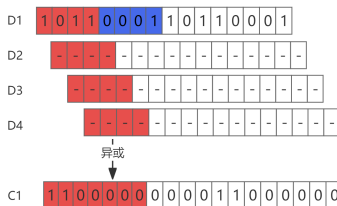
原来的问题

长块直接遍历时，数据块和校验块之间来回跳转，相关数据还没用完就可能离开缓存。

FastTT 的做法

- 把每个长块划分为连续窗口
- 当前窗口内完成移位和异或
- 首次访问时再初始化，避免整块预清零

chunk size = 16, window size = 4, D1只代入第1个窗口，无法覆盖D2、D3和D4的第1个窗口
解决方案：off-by-one window, D1领先代入1个窗口，即可覆盖，同时几乎不影响窗口局部性



同色片段表示同一窗口内的局部访问。



已完成工作与结果

相邻数据先合并，校验块少写一半

移位异或纠错码的
高效实现研究

孙铎

目录

研究内容与进度

已完成工作与结果

后期工作、困难与可行性

原来的问题

横向遍历虽然连续读数据，但每个数据块都要分散写入多个校验块，写放大明显。

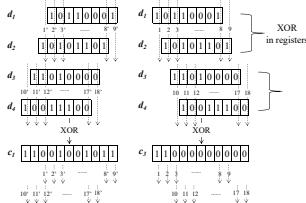
FastTT 的做法

相邻两个数据块先在寄存器内异或合并，再一次性写回校验块。

直接结果

校验块写入访问量由约 km 降至约 $km/2$ 。

合并后：相邻数据合并，再连续写回校验块





已完成工作与结果

单块丢失最常见，直接走快路径

移位异或纠删码的
高效实现研究

孙铎

目录

研究内容与进度

已完成工作与结果

后期工作、困难与可行性

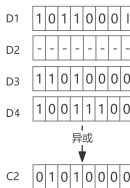
为什么值得单独优化

- 大规模存储中单块失效最常见
- 单块丢失不需要完整 Two-tone 消除
- 直接对相关存活块做一遍异或即可恢复

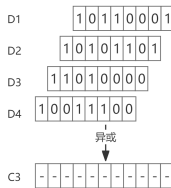
直接结果

- 单块丢失相较 ISA-L 提升超过 130%
- 最高提升达到 182.1%

只丢失一个数据块



只丢失一个校验块



多块丢失时自动回到标准统一解码流程。



已完成工作与结果

结果一：编码吞吐明显高于 Ceph 现有基线

移位异或纠删码的
高效实现研究

孙铎

目录

研究内容与进度

已完成工作与结果

后期工作、困难与可行性

最重要的数

+59.1%

相较 Ceph 生产级基线 ISA-L
的平均编码吞吐提升

实验怎么测

- Ceph v18.2.7 纠删码插件
- Intel Core i9-14900K, 启用 AVX2
- 块大小 2 MB, 每组重复 2000 次

编码吞吐 (GB/s)

配置	FastTT	ISA-L	Jerasure
(6, 2)	34.74	24.37	17.03
(6, 3)	31.69	17.59	7.64
(8, 3)	31.35	17.03	7.08
(10, 4)	17.59	13.56	4.58

即使在 (10, 4) 这种对寄存器更不友好的配置下, FastTT 仍保持领先。



已完成工作与结果

结果二：解码优势在单块丢失时最明显

移位异或纠删码的
高效实现研究

孙铎

目录

研究内容与进度

已完成工作与结果

后期工作、困难与可行性

最重要的数

+50.9%

平均解码吞吐提升

+182.1%

单块丢失最高提升

怎么看表

丢失块数为 1 时，快路径生效；两块及以上丢失时，回到完整统一解码流程。

解码吞吐 (GB/s)

配置	丢失	FastTT	ISA-L	Jerasure
(6,2)	1	95.99	38.38	42.70
(6,2)	2	25.60	24.89	16.26
(6,3)	1	95.18	39.43	44.57
(6,3)	2	24.24	24.77	12.12
(6,3)	3	19.18	18.16	5.39
(8,3)	1	93.11	33.01	37.04
(8,3)	2	26.18	23.16	10.97
(8,3)	3	21.44	18.38	5.00
(10,4)	1	81.01	34.42	35.11
(10,4)	2	20.96	22.23	10.62
(10,4)	3	15.26	17.08	4.73
(10,4)	4	11.53	13.97	3.69



已完成工作与结果

结果三：相较学术基线 Cerasure 仍有优势

移位异或纠删码的
高效实现研究

孙铎

目录

研究内容与进度

已完成工作与结果

后期工作、困难与可行性

对比方式

Cerasure 不直接集成 Ceph，
因此按其原始测试方法对比，
保证与学术基线的实验口径
一致。

结果摘要

- 编码平均提升 13.2%
- 解码平均提升 16.0%
- 解码最高提升 43.6%
- 个别两块丢失场景略低

编码吞吐 (GB/s)

配置	FastTT	Cerasure
(6, 2)	47.97	44.20
(6, 3)	32.01	30.97
(8, 3)	32.75	28.72
(10, 4)	24.58	19.36

解码吞吐 (GB/s)

配置	丢失	FastTT	Cerasure
(6,2)	1	88.41	76.22
(6,2)	2	46.80	43.08
(6,3)	1	87.69	75.45
(6,3)	2	39.40	41.20
(6,3)	3	21.09	17.82
(8,3)	1	82.59	74.91
(8,3)	2	39.28	39.94
(8,3)	3	24.14	16.81
(10,4)	1	73.95	69.89
(10,4)	2	36.66	35.68
(10,4)	3	21.59	15.60
(10,4)	4	16.93	12.27

孙铎

哈尔滨工业大学（深圳）计算机科学与技术学院

移位异或纠删码的高效实现研究

15 / 19



已完成工作与结果

消融实验：四个改动都带来实际收益

移位异或纠删码的
高效实现研究

孙铎

目录

研究内容与进度

已完成工作与结果

后期工作、困难与可行性

从原始实现到完整实现

- 边解码边修校验：解码提升约 100.2%
- 窗口划分：编解码继续提升约 43.0%
- 相邻合并：编码提升约 58.6%
- 单块快路径：解码提升约 43.5%

消融实验 (8,3) (GB/s)

窗口	合并	边修校验	快路径	编码	解码
				13.16	7.54
		✓		13.16	16.85
✓		✓		23.32	24.10
✓	✓	✓		31.23	24.10
✓	✓	✓	✓	32.78	35.23

完整实现

相较原始实现，编码提升约 149%，
解码提升约 367%。



后期工作、困难与可行性

后面还要补什么证据

移位异或纠删码的
高效实现研究

孙铎

目录

研究内容与进度

已完成工作与结果

后期工作、困难与可行性

孙铎

移位异或纠删码的高效实现研究

围绕现有 FastTT 补机理实验

- 测缓存命中率：解释窗口划分收益
- 统计内存读写次数：量化写放大下降
- 对比“解码后再修复”和“边解码边修复”
- 统计单块快路径的每码字访问次数

围绕论文和毕业设计补材料

- 根据论文审稿意见补充实验和分析
- 整理面向移位异或码实现的经验总结
- 完成毕业论文正文与最终答辩材料

新增创新点方向

大语言模型推理会产生大量键值缓存 (KV Cache)。现有缓存池常用多副本保证可用性，占用内存高。

后续探索用 FastTT 的高吞吐纠删冗余替代部分副本，在控制恢复延迟的同时提升等效缓存容量。

第一步验证

做小规模原型，比较“恢复 KV 块”和“重新预填充计算”的延迟边界。

哈尔滨工业大学（深圳）计算机科学与技术学院

17 / 19



后期工作、困难与可行性

困难已经明确，后续工作边界清楚

移位异或纠删码的
高效实现研究

孙铎

目录

研究内容与进度

已完成工作与结果

后期工作、困难与可行性

主要困难与对应动作

- (10, 4) 配置受 AVX2 寄存器数量影响 $\Rightarrow$ 尝试更灵活的分组和更宽向量指令
- 两块丢失场景无法使用单块快路径 $\Rightarrow$ 分析访问模式，寻找中间优化策略
- 当前优化主要围绕 Two-tone 结构 $\Rightarrow$ 抽取与具体偏移矩阵无关的通用部分
- 键值缓存方向仍有收益不确定性 $\Rightarrow$ 先验证条带粒度、恢复延迟和缓存命中收益

为什么能按期完成

- 核心算法实现已完成
- Ceph 插件和正确性测试已完成
- 编码、解码、消融和基线对比实验已完成
- 论文投稿材料已经形成
- 后续主要是补证据、写论文和做新增方向原型

能够按期完成。



Q&A

移位异或纠删码的
高效实现研究

孙铎

目录

研究内容与进度

已完成工作与结果

后期工作、困难与可行性

感谢各位老师的耐心观看
请老师批评指正

Q&A